Assignment 2 Q3(b): Proposed algorithm design

Q3(b): RTT, throughput, and loss based window control

Status: proposed design. The completed q3-test dataset compares CUBIC, Reno, and BBR at three destinations with three 60-second repetitions each. Those measurements informed the design rationale below; the proposed algorithm itself has not been implemented or measured. Assignment 3 must implement and evaluate it separately.

State and measurements

Maintain cwnd and ssthresh in segments, initialized to 10 and infinity; obey any lower kernel or peer limit. Minimum cwnd is 2. Let MSS be the current sending maximum segment size in bytes. Use the host TCP stack for ACK processing, retransmission, recovery, and timeout handling.

Update once per measured RTT, using only newly acknowledged data. During each epoch collect acknowledged bytes, elapsed time, RTT samples, and newly retransmitted segments. Goodput is acknowledged bytes / elapsed seconds, in bytes/s. Estimate throughput with an EWMA: rate = 0.75 * previous_rate + 0.25 * epoch_goodput; initialize it from the first valid epoch.

Let base_rtt be the minimum valid RTT in a rolling 10-second window. Let queue_ratio = max(0, srtt / base_rtt - 1). Keep a rolling maximum of non-application-limited epoch goodput over 10 seconds as bandwidth_estimate. Then bdp_segments = max(2, bandwidth_estimate * base_rtt / MSS). RTT values in this calculation are seconds. This is an observed delivery-rate estimate, not a measurement of physical link capacity.

A loss event means a newly entered TCP recovery episode, not every ACK that reports retransmissions. Only reduce once in that episode, respecting the kernel recovery boundary. Ignore epochs with no newly acknowledged bytes for growth; timeout handling still applies. Freeze delay and throughput growth decisions when the sender is application limited or receiver-window limited.

Assignment 2 Q3(b): Proposed algorithm design

Pseudocode

```
State: cwnd = 10; ssthresh = infinity; mode = STARTUP; rate = unset
On valid ACK: accumulate delivered bytes and RTT samples for this epoch
On entering a new loss recovery episode:
    cwnd = max(2, floor(0.70 * cwnd))
    ssthresh = cwnd; mode = AVOIDANCE
    mark recovery boundary; reset epoch accounting
On retransmission timeout:
    ssthresh = max(2, floor(cwnd / 2))
    cwnd = 2; mode = AVOIDANCE; reset epoch accounting
Once per RTT, after ACK progress, outside loss recovery:
    update base_rtt, bandwidth_estimate, and rate
    if application limited or receiver-window limited: hold cwnd; finish epoch
    if mode == STARTUP:
        if queue_ratio < 0.10:
            cwnd = cwnd + min(newly_acked_segments, cwnd)
            finish epoch
        mode = AVOIDANCE; ssthresh = cwnd
    if queue_ratio > 0.25:
        cwnd = max(2, floor(0.90 * cwnd))
    else if queue_ratio < 0.10:
        if cwnd < 1.25 * bdp_segments or rate > 1.02 * previous_rate:
            cwnd = cwnd + 1
        else if at least 8 RTT epochs have passed since the last probe:
            cwnd = cwnd + 1
            reset probe timer
    else:
        hold cwnd
    enforce kernel cwnd clamp; finish epoch
```

Assignment 2 Q3(b): Proposed algorithm design

Initialize previous_rate from the first epoch and skip the rate-growth comparison for that epoch. Count newly acknowledged segments with fractional credit across ACKs, so delayed ACKs and partial segments do not inflate growth. Maintain the probe timer across epochs, incrementing it once per eligible RTT and resetting it on a probe or reduction. Do not stack a delay reduction on a loss reduction within the same recovery epoch. A kernel/eBPF implementation must let the stack apply timeout behavior exactly once rather than performing a second reduction in an ACK callback.

Design choices

The existing CUBIC traces motivated retaining acknowledged-byte goodput, smoothed RTT, and retransmission counters as the observable inputs. Two 15-second runs to 185.180.12.40:5201 had duration-weighted goodputs of 164.20 and 93.62 Mbit/s, mean smoothed RTTs of 128.18 and 129.08 ms, and retransmission deltas totaling 0 and 147 segments. Their median congestion windows were 4165 and 1135.5 segments. These runs used the same algorithm: they show substantial variability and motivate measuring throughput and loss alongside RTT, rather than inferring useful rate from cwnd alone. They do not establish that retransmissions caused the entire goodput difference, and they do not establish optimal thresholds. The source files are results/20261018T035444Z_185.180.12.40_5201.csv and results/20261018T035817Z_185.180.12.40_5201.csv; calculations use run_statistics in comparison_plots.py. The 10% and 25% queue thresholds create a band in which the sender holds its window. Low delay allows probing; sustained queue growth causes a small reduction before loss. Loss triggers a larger reduction. The eight-RTT probe avoids permanently accepting an underestimated bandwidth estimate after a path gains capacity. The BDP gate limits routine growth when extra window stops increasing useful delivery. Startup provides initial discovery without an established bandwidth estimate.

Assignment 2 Q3(b): Proposed algorithm design

The full baseline comparison reinforces the use of all three signals. In Dallas, Reno's median cwnd rose from 1,982 to 7,854 segments across repetitions while goodput stayed near 279-284 Mbit/s, motivating a throughput gate on routine window growth. On Netfiber, BBR's third run fell to 86.57 Mbit/s with 396 retransmissions despite mean RTT near the path baseline, motivating a separate loss response rather than relying on RTT alone. At 96.45.41.167, one BBR interval had zero acknowledged goodput and a 1,559.64 ms smoothed RTT, motivating a conservative response to sustained delay inflation as well. These are associations in sequential public-path tests, not proof of any mechanism or validation of the proposed thresholds. The analysis and source measurements are in results/q3-test/observations.md and its referenced CSV files.

The constants remain initial hypotheses. We retain the previously shared pseudocode and thresholds so this handoff stays consistent; the new data do not justify presenting those constants as tuned or optimal. Validate them in controlled tests and document any changes in Assignment 3. A rolling base RTT adapts after route changes but can mistake a persistent queue for propagation delay when old samples expire. A rolling bandwidth maximum also lags rapid capacity decreases.

Fairness and stability

Under equal RTTs, persistent backlog, similar packet sizes, and a shared drop-tail bottleneck, equal additive growth with proportional loss reductions can move flows toward similar windows. The queue response can reduce the amplitude of a congestion cycle. This is an expectation, not a proof of fairness or convergence.

Flows with different RTTs update at different frequencies, so RTT fairness is not guaranteed. A delay-sensitive flow may yield to CUBIC or Reno flows that maintain queues. Its estimated minimum RTT may include standing queues, and independent bandwidth estimates can overstate each flow's fair share. The throughput gate can also preserve an unfair initial allocation. Public Internet measurements cannot isolate these effects.

For a single flow on a fixed path, the hold band and rate smoothing should reduce rapid changes, but periodic probing and multiplicative reductions can produce a limit cycle rather than a fixed window. Delayed feedback, ACK compression, reverse-path congestion, random wireless losses, and changing routes can destabilize decisions. Evaluate equal and unequal RTT competing flows, utilization, queue delay, recovery, and Jain's fairness index before claiming stable or fair operation.

Assignment 2 Q3(b): Proposed algorithm design

Assignment 3 handoff

Implement this state machine in a kernel congestion-control module or eBPF struct_ops program. A Python client cannot implement a kernel congestion window policy by selecting TCP_CONGESTION alone. Use kernel delivery/rate samples and ACK/recovery callbacks to supply the inputs above. Test that callbacks preserve the once-per-RTT and once-per-recovery rules. Record callback mappings, unavailable signals, parameter changes, exact kernel version, and the actual loaded algorithm name. The comparison command accepts extra algorithm names, so it can evaluate the custom algorithm alongside cubic, reno, and bbr with 10-second runs.